

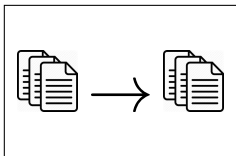


Introduction to PyTorch

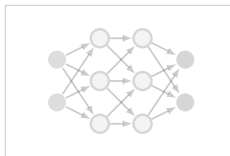
Deep Learning for Scientific Computing

Robert Jendersie

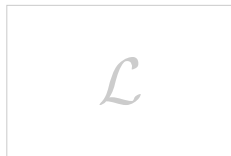
Ingredients



Problem

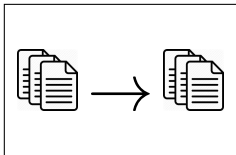


Neural Network

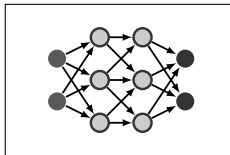


Optimizer

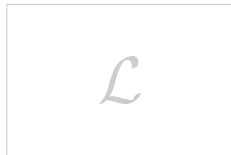
Ingredients



Problem

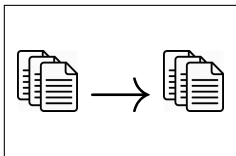


Neural Network

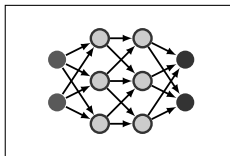


Optimizer

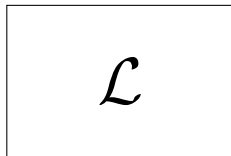
Ingredients



Problem

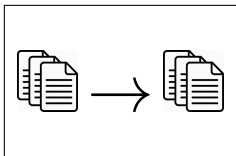


Neural Network

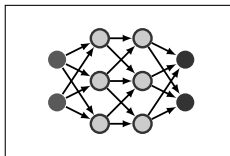


Optimizer

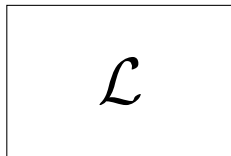
Ingredients



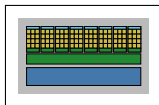
Problem



Neural Network



Optimizer



GPU

Machine Learning Frameworks



Machine Learning Frameworks



PyTorch

- De facto standard for deep learning



Machine Learning Frameworks



PyTorch

- De facto standard for deep learning
- Most commonly used primitives built-in



Machine Learning Frameworks



PyTorch

- De facto standard for deep learning
- Most commonly used primitives built-in



- Framework for differentiable programming

Machine Learning Frameworks



PyTorch

- De facto standard for deep learning
- Most commonly used primitives built-in



- Framework for differentiable programming
- Supports TPUs

Problem Description

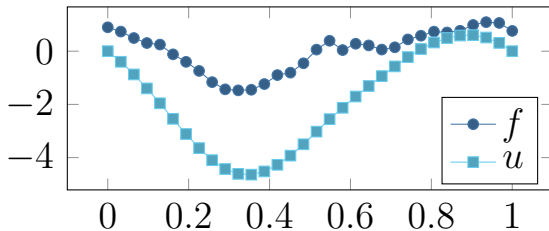
$$\begin{aligned} -u''(x) &= f(x), x \in [0, 1], \\ u(0) &= u(1) = 0, \end{aligned}$$

$$f(x) = \sum_{k=1}^K a_k \sin(2\pi kx) + b_k \cos(2\pi kx), \quad a_k, b_k \sim \mathcal{N}(0, 1).$$

Problem Description

$$\begin{aligned}-u''(x) &= f(x), x \in [0, 1], \\ u(0) &= u(1) = 0,\end{aligned}$$

$$f(x) = \sum_{k=1}^K a_k \sin(2\pi kx) + b_k \cos(2\pi kx), \quad a_k, b_k \sim \mathcal{N}(0, 1).$$



Artificial Neural Network

MLP : $\mathbb{R}^{N_{\text{in}}} \rightarrow \mathbb{R}^{N_{\text{out}}}$ with

$$\text{MLP}(x) := (g_L \circ \cdots \circ g_1)(x)$$

with layers $g_i : \mathbb{R}^{N_i} \rightarrow \mathbb{R}^{N_{i+1}}$, where

$$g_j(x) = \sigma(W_j x + b_j), \quad j = 1, \dots, L-1$$

$$g_L(x) = W_L x + b_L.$$

learned weight matrix $W_i \in \mathbb{R}^{N_i \times N_{i+1}}$ and bias $b_i \in \mathbb{R}^{N_{i+1}}$

Universal Approximation Theorem¹

Let $K \subseteq \mathbb{R}^n$ be compact. Let $\mathbf{M}$ denote the space of functions $\text{MLP} : \mathbb{R}^{N_{\text{in}}} \rightarrow \mathbb{R}^{N_{\text{out}}}$ of the form

$$\text{MLP}(x) = W_2 \sigma(W_1 x + b),$$

where $W \in \mathbb{R}^{N_{\text{hid}} \times N_{\text{in}}}$, $b \in \mathbb{R}^{N_{\text{hid}}}$, $W \in \mathbb{R}^{N_{\text{out}} \times N_{\text{hid}}}$ and $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ is continuous.

¹Leshno et al., "Multilayer feedforward networks with a nonpolynomial activation function can approximate any function", 1993.

Universal Approximation Theorem¹

Let $K \subseteq \mathbb{R}^n$ be compact. Let $\mathbf{M}$ denote the space of functions $\text{MLP} : \mathbb{R}^{N_{\text{in}}} \rightarrow \mathbb{R}^{N_{\text{out}}}$ of the form

$$\text{MLP}(x) = W_2 \sigma(W_1 x + b),$$

where $W \in \mathbb{R}^{N_{\text{hid}} \times N_{\text{in}}}$, $b \in \mathbb{R}^{N_{\text{hid}}}$, $W \in \mathbb{R}^{N_{\text{out}} \times N_{\text{hid}}}$ and $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ is continuous. Then, if and only if σ is not a polynomial, there exists a $\text{MLP} \in \mathbf{M}$ for any continuous function $f : K \rightarrow \mathbb{R}$ and every $\epsilon > 0$ such that

$$\sup_{x \in K} \|\text{MLP}(x) - f(x)\| < \epsilon.$$

¹Leshno et al., "Multilayer feedforward networks with a nonpolynomial activation function can approximate any function", 1993.

Optimizer

mini-batch (f, u) with $f, u \in \mathbb{R}^{B \times N}$

Optimizer

mini-batch (f, u) with $f, u \in \mathbb{R}^{B \times N}$

$$\mathcal{L}(u, \hat{u}) = \frac{1}{BN} \sum_{i=1}^B \sum_{j=1}^N (u_{ij} - \hat{u}_{ij})^2$$

Optimizer

mini-batch (f, u) with $f, u \in \mathbb{R}^{B \times N}$

$$\mathcal{L}(u, \hat{u}) = \frac{1}{BN} \sum_{i=1}^B \sum_{j=1}^N (u_{ij} - \hat{u}_{ij})^2$$

$$w_{r+1} = w_r - \eta \frac{\partial}{\partial w} \mathcal{L}(u, \text{MLP}(f; w_r)),$$

with learning rate $\eta > 0$

Getting Started

- Clone repository

```
git clone https://github.com/Thanduriel/pytorch-workshop.git
```

- Create Python virtual environment

```
python -m venv pytorch-env  
source pytorch-env/bin/activate
```

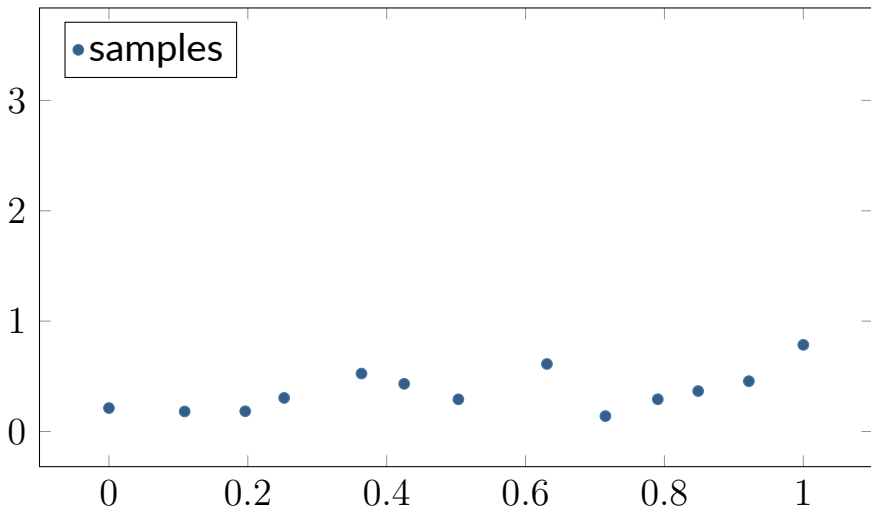
- Install PyTorch (<https://pytorch.org/get-started/locally/>)

```
pip install torch --index-url \\\nhttps://download.pytorch.org/whl/cpu
```

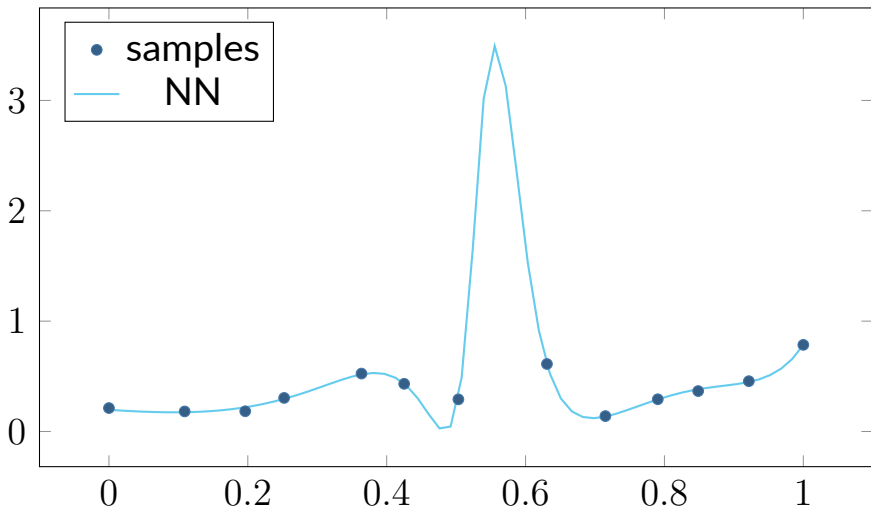
- Install other Python packages

```
pip install -r requirements.txt
```

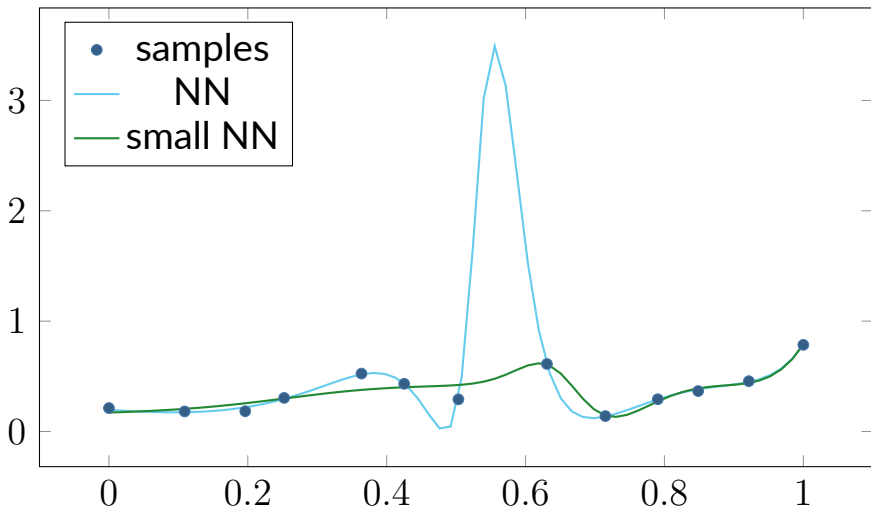
Regularization



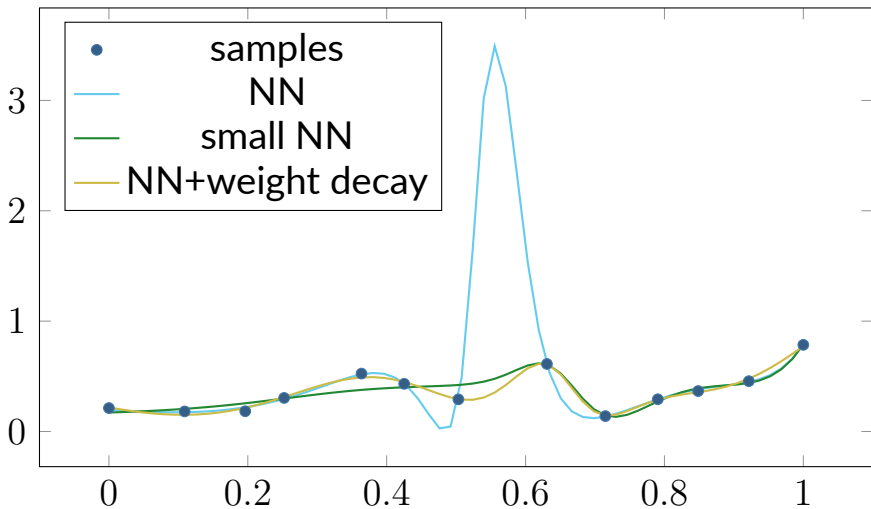
Regularization



Regularization



Regularization



Regularization Techniques

Weight decay (Tikhonov regularization)

$$\hat{\mathcal{L}}(u, \hat{u}, w_r) = \mathcal{L}(u, \hat{u}) + \lambda ||w_r||_2^2$$

Regularization Techniques

Weight decay (Tikhonov regularization)

$$\hat{\mathcal{L}}(u, \hat{u}, w_r) = \mathcal{L}(u, \hat{u}) + \lambda ||w_r||_2^2$$

Layer normalization

$$\text{LN}(x) = \frac{x - \mathbf{E}[x]}{\mathbf{Var}[x]} \odot \gamma + \beta$$

Further Reading

- Intuitive explanation of generalization abilities of SGD

W. Ronny Huang et al. “Understanding Generalization through Visualizations”. In: (June 2019). DOI: 10.48550/ARXIV.1906.03291. arXiv: 1906.03291 [cs.LG]

- AdamW and weight decay

Ilya Loshchilov and Frank Hutter. “Decoupled Weight Decay Regularization”. In: (Nov. 2017). DOI: 10.48550/ARXIV.1711.05101. arXiv: 1711.05101 [cs.LG]

- Normalization techniques

L. Huang et al. “Normalization Techniques in Training DNNs: Methodology, Analysis and Application”. In: IEEE Transactions on Pattern Analysis and Machine Intelligence 45.8 (2023), pp. 10173–10196. DOI: 10.1109/tpami.2023.3250241